AI
ENGINEERING
AI-Powered products

MÓDULO 3: Data-driven AI

Sesión 7: Embeddings y representación vectorial

Mentor: Julián Fernández
LIDR.co, training the Tech Leaders' of the future

L1DR

Tres arquitecturas, una base común

Por qué reorganizamos el proyecto en esta sesión · de carpetas planas a arquitectura por capas

ESTRUCTURA DEL PROYECTO

```
app/
├── foundation/        plomería sin opinión
├── domain/            schemas + el "conductor"
├── generation/        las 3 arquitecturas que COMPONEN
│   ├── cag/           CAG   caches (ya construido)
│   ├── rag/           RAG   ● LO DE HOY
│   │   ├── chunking/ · embedding/ · analysis/
│   │   ├── store/         ← slot reservado S8
│   │   └── retriever.py   ← slot reservado S8
│   ├── agentic/       AGENTIC   boss + critic (futuro)
│   └── conversation/
├── ingestion/         pipeline batch que alimenta RAG
└── api/               transporte (routers finos)
```

REGLA: Los hermanos de generation/ no se importan entre sí · solo se encuentran en el conductor

Qué cambia exactamente

El estimador apila tres arquitecturas de IA de forma aditiva a lo largo del máster — CAG, RAG (hoy) y agentic (futuro).

A partir de hoy esa historia se ve en la estructura.

Por qué ahora

Las carpetas planas escondían el patrón: caches partidas, RAG aislado, lo agéntico mezclado en services.

Antes de añadir más piezas, ponemos cada cosa en su capa.

Lo de hoy en este mapa

Todo lo que tocamos en la S7 es exactamente el contenido de **`app/generation/rag/`**

store/ y retriever.py se rellenan en la S8.

Frase para defender ante un stakeholder

"No reescribimos lógica: movimos las piezas a su capa. La arquitectura ahora se lee de un vistazo y cada cosa nueva sabe dónde va."

L1DR

Cinco patrones de error reales

Ordenados por impacto en producción

1. **Sin validación Pydantic**
   Síntoma: `KeyError` opaco en producción cuando falta un campo
   Resuelto en · app/generation/rag/schemas.py

2. **Gestión naïve de tokens**
   Síntoma: chunk excede contexto del modelo · `422` de la API de embeddings
   Resuelto en · app/generation/rag/chunking/base.py :: count_tokens

3. **Sin batching**
   Síntoma: un request por chunk · latencia que se dispara · throughput inviable
   Resuelto en · embedder.py :: BATCH_SIZE = 100 + embed_many

4. **Sin backoff exponencial**
   Síntoma: caída ante el primer `RateLimitError` · sin resiliencia
   Resuelto en · embedder.py :: _RETRY_BACKOFF_SECONDS = (1, 2, 4)

5. **Dimensionalidad inconsistente**
   Síntoma: mezclar vectores de 1536 y 768 en el mismo índice rompe la similitud
   Resuelto en · schemas.py :: EmbeddedChunk con campo embedding tipado

Pregunta gancho del bloque

"Vuestro chunker no truncó
esa descripción de 1.391 tokens.
¿Está bien o está mal?"

Respuesta esperada: depende.
Hoy no rompe. Mañana sí.

L1DR

Matryoshka: la palanca de dimensiones

Mismo modelo · misma señal · la mitad de storage

Modelo	Dimensiones	Latencia media	Cosine(c0, c1)	Δ Calidad
`text-embedding-3-small` @ 1536d nativo	1536	879.8 ms	0.9119	baseline
`text-embedding-3-small` @ 768d (Matryoshka)	768	699.9 ms	0.9117	–0.0002

½ storage

Mitad de dimensiones = mitad de bytes por vector en la BBDD

Para 1M de chunks: GB que entran en RAM o que no entran

Sin pérdida real

Cosine entre dos componentes similares: idéntica hasta el cuarto decimal

0.9119 vs 0.9117 — ruido

Un parámetro

Truncar dimensiones no es un modelo nuevo. Es el argumento **`dimensions=768`**

Palanca futura para S8

LOCKED PARA EL PROYECTO

text-embedding-3-small @ 1536d · API ya configurada · coste despreciable · Matryoshka como palanca futura

L1DR

Las cuatro familias de chunking

Ocho estrategias agrupadas por cómo deciden dónde partir el texto

1 Mecánicas

Cortan por reglas · ignoran el contenido

Bloque 4 — 30 min

- FixedSize — baseline degenerado
- Recursive — default razonable
- SentenceWindow — oración indexada · ventana devuelta

Coste: $0

2 Estructurales

Explotan el formato del documento

Bloque 4 (parcial) + Bloque 6

- JSONStructural — un componente = un chunk
- Hierarchical — parent-child · dos niveles

Coste: $0

3 Semánticas

Cortan donde cambia el significado

Bloque 5 — primera mitad

- Semantic — breakpoint por caída de similitud
- Propositional — LLM descompone en proposiciones

Coste real · medir ROI

4 Contextuales

Enriquecen el chunk antes de embedear

Bloque 5 + Bloque 6

- ContextualRetrieval — Claude + prompt caching
- LateChunking — demo conceptual

Coste premium

Todas implementan la misma interfaz Chunker · intercambiables en el comparador

L1DR

Fixed-size vs Recursive

Mismo presupuesto serializado · dos formas radicalmente distintas de partirlo

FixedSize · CHUNK_SIZE_TOKENS = 512 · OVERLAP = 80

BUD-2024-016::MONO-001::p0 · 512 tokens

```
[Project: Migration of legacy banking platform...] Component:
Core banking monolith decomposi tion into 18 microservices
Description: Strangler-fig migration of a 12-year-old Java mono
lith handling 2.4M daily transactions. The migration plan spa
ns 18 months and inclu
```

BUD-2024-016::MONO-001::p1 · 512 tokens

```
des the following dom ains: account management, transactional
core, identity services, fraud detection, audit, regulatory r
eporting, KYC compliance, AML monitoring, customer profiling,
notification dispatch, scheduled batches, and external integ
rations with PSD2-com
```

BUD-2024-016::MONO-001::p2 · 367 tokens

```
pliant providers. Each domain will be implemented as an in
dependent service with its own database and event bus...
[...]
```

63 chunks · p50: 79 tok · max: 512 tok · obesos: 0

"decomposi tion" · "dom ains" · palabras rotas a mitad

Recursive · separadores [\n\n, \n, ". ", " ", ""]

BUD-2024-016::r0 · 488 tokens

```
[Project: Migration of legacy banking platform to cloud-native
microservices architecture]

Component: Core banking monolith decomposition into
18 microservices.
Description: Strangler-fig migration of a 12-year-old Java...
```

BUD-2024-016::r1 · 494 tokens

```
...handling 2.4M daily transactions.

The migration plan spans 18 months and includes the
following domains: account management, transactional core,
identity services, fraud detection, audit, regulatory
reporting, KYC compliance...
```

BUD-2024-016::r2 · 187 tokens

```
...AML monitoring, notification dispatch, scheduled batches,
and external integrations with PSD2-compliant providers.
```

23 chunks · p50: 187 tok · max: 494 tok · obesos: 0

Oraciones intactas · respeta saltos naturales del lenguaje

El default razonable: Recursive con 400–512 tokens y 10–20% de overlap

Solo se complejiza con evidencia medida de que no es suficiente · MVP con un día de plazo: este es el chunker

L1DR

Sentence-window vs Structural

Misma consulta · resultados que parecen iguales pero esconden la trampa del score alto

CONSULTA: **"OAuth authentication for fintech mobile app"**

SentenceWindow · indexa oraciones sueltas

#1 [0.4530] BUD-2024-001::PSD2-002::s0
"Implements PSD2 strong customer authentication..."
relevante

#2 [0.4410] BUD-2024-017::MVP-001::s0
"Email and password login."
⚠ del MVP de e-commerce — score alto, contexto perdido

#3 [0.4110] BUD-2024-006::VENDOR-001::s0
"Third-party vendor onboarding with SSO..."
parcialmente relevante

JSONStructural · un componente = un chunk

#1 [0.5684] BUD-2024-001::AUTH-001
[Project: Mobile banking API...] OAuth 2.0 backend...
✓ el componente OAuth correcto con contexto

#2 [0.5310] BUD-2024-001::NOTIF-004
[Project: Mobile banking API...] Notification svc...
mismo proyecto · relacionado

#3 [0.5220] BUD-2024-001::PSD2-002
[Project: Mobile banking API...] PSD2 SCA module...
✓ relacionado, mismo dominio

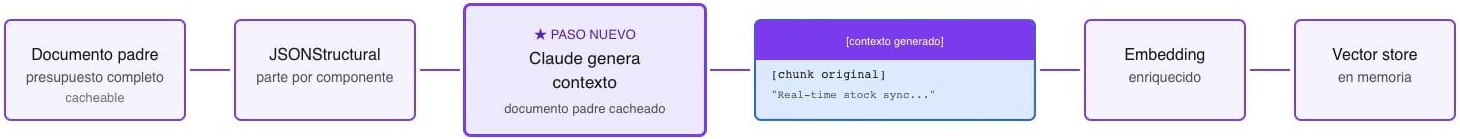
Lo que el sentence-window oculta

Indexa oraciones diminutas y descontextualizadas · "Email and password login." parece auth, no lo es
La oración recuperada NO es lo que pasarías al LLM-generador · eso vive en **`metadata.window_text`**
Score alto ≠ chunk relevante. La unidad indexada debe ser distinta de la devuelta cuando la indexada pierde sentido aislada.

L1DR

Contextual Retrieval

Enriquece el chunk con un párrafo generado por Claude · prompt caching del documento padre



RESULTADO SOBRE NUESTRO CORPUS · consulta: "OAuth authentication for fintech mobile app"

JSONStructural (baseline) Top-1: BUD-2024-001::AUTH-001	0.5684	
ContextualRetrieval Mismo chunk + contexto generado por Claude	0.6070	+0.04

COSTE REAL · 60 chunks · 17 presupuestos

JSONStructural		$0.00 · 0 segundos
ContextualRetrieval		$0.140 · 202 segundos (3.4 min)

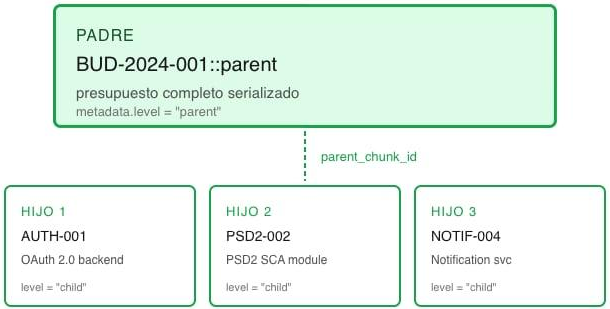
+0.04 de cosine por $0.14 · 3.4 minutos de ingesta · ¿lo aprobáis en una review?

Anthropic reporta hasta –67% de fallos de retrieval combinado con BM25 contextual + reranking

El header estático del JSONStructural ya es Contextual Retrieval barato · la versión LLM solo paga la pena en corpus donde compensa medir

Hierarchical · parent-child

Dos niveles indexados a la vez · el hijo da el recall, el padre da el contexto



CONSULTA: **"GDPR compliance and audit logging"**

#1 HIJO [0.4180] `BUD-2024-009::CONSENT-004`

`[Project: Healthcare data platform...] Consent and audit log...`

↓ con --resolve-parents activado, se devuelve también el padre BUD-2024-009::parent

#2 PADRE [0.3070] `BUD-2024-012::parent`

`presupuesto completo serializado`

⚠ un padre entero entra en el top-3 sin necesidad de resolver

Hijos para recall

Granularidad fina · matchea
consultas específicas con score alto.

El embedding del componente es preciso y discriminante.

Padres para contexto

Cuando el hijo matchea, se
devuelve también el padre completo.

Útil para queries abstractas: sube el padre al top-3 por sí solo.

¿Cuándo activar?

✓ Manuales largos
✓ Documentos legales
✗ FAQs cortos
✗ Tickets breves · presupuestos: borderline

Stats: 77 chunks = 17 padres + 60 hijos · obesos: 2 (max 1472) · coste: $0

L1DR

Lo que cierras hoy · lo que abre la S8

SALIMOS HOY CON

- Ocho estrategias ejecutadas sobre tu corpus
- Métricas comparativas: stats + cosine + cualitativa
- Una estrategia elegida con justificación
- Plan de qué medirías antes de cambiarla
- 60 chunks vectorizados · en memoria

EN MEMORIA · HTTP

LA SESIÓN 8 AÑADE

- PostgreSQL + pgvector
- Índices HNSW / IVFFlat para escala
- Filtrado por metadata + similitud combinada
- Búsqueda semántica real a baja latencia
- Persistencia que sobrevive al restart

PERSISTENCIA · ÍNDICES

EL ARCO COMPLETO DEL MÓDULO

SESIÓN 06
Corpus limpio
→
SESIÓN 07 ✓
Chunking + vectores
→
SESIÓN 08 ← siguiente
pgvector + retrieval
→
SESIÓN 10
Retrieval avanzado
→
SESIÓN 11
Generación y calidad

El código de hoy se queda intacto · la S8 añade una capa, no sustituye nada

✋Tiempo para el feedback con ROTI

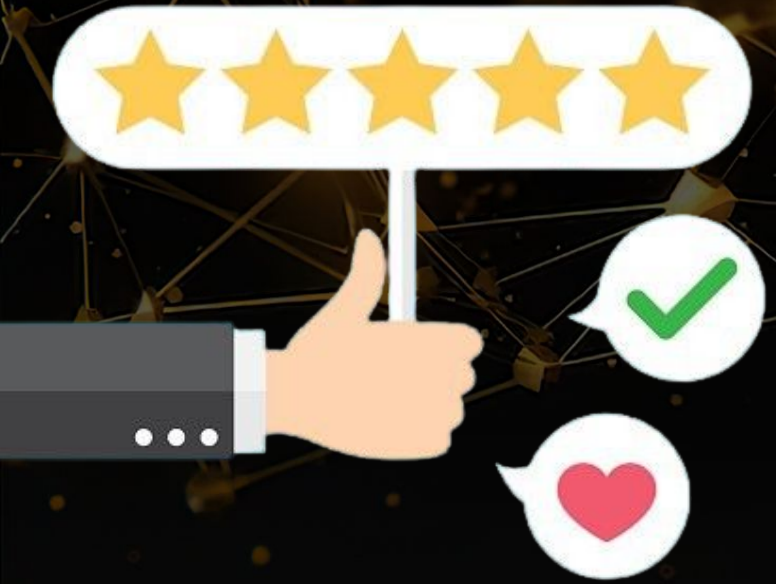
¿Cuánto valor te ha agregado esta sesión?



Absolutamente nada	Poco	Suficiente	Bastante	Más de lo que esperaba

LIDR

ROTI



¡GRACIAS!

LIDR.co, AI-Powered Careers

@lidr_ai

#followtheLIDR